Reviewer Pack — Communication Matrix Discrepancy Lower Bound for AC⁰ PARITY ...

Entry dd369e80c960 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 12:50:05 UTC

Communication Matrix Discrepancy Lower Bound for AC⁰ PARITY Circuits

Entry ID: `dd369e80c960` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 12:50:05 UTC

1. Conjecture

Field A (mathematical branch): Communication Complexity **Field B** (complexity object): AC⁰ Circuit Complexity

Statement:

> For any AC⁰ circuit C computing PARITY on n inputs, the discrepancy of its communication matrix M_C satisfies disc(M_C) ≥ c · log(size(C)) for some absolute constant c > 0.

Rationale (proposer's reasoning):

> The discrepancy of the communication matrix for PARITY is tightly linked to its communication complexity, which is known to be Ω(log n) for AC⁰ circuits. By leveraging the invariance of discrepancy under variable permutations (a non-trivial group action), this invariant provides a natural measure of circuit complexity that replaces the random restriction argument in switching lemmas.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9c5cc34d909085c4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import product

def generate_ac0_circuit(n, depth):
    if n == 1:
        return [random.randint(0, 1) for _ in range(depth)]
    subcircuits = [generate_ac0_circuit(n // 2, depth - 1) for _ in range(2)]
    return [subcircuits[0][i] ^ subcircuits[1][i] for i in range(n)]

def hadamard_transform(matrix):
    n = len(matrix)
    result = [[0] * n for _ in range(n)]
    for i, j in product(range(n), repeat=2):
        result[i][j] = (matrix[i // 2][j // 2] if i % 2 == j % 2 else -matrix[i // 2][j // 2]) / m
    return result

def discrepancy(matrix):
    n = len(matrix)
    transform = hadamard_transform(matrix)
    max_discrepancy = 0
    for row in transform:
        max_discrepancy = max(max_discrepancy, sum(abs(x) for x in row))
    return max_discrepancy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    depth = random.randint(1, 3)
    circuit = generate_ac0_circuit(n, depth)
    size = len(circuit)
    disc_value = discrepancy([circuit])
    conjecture_holds = disc_value >= math.log(size) / math.log(2)
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "discrepancy",
        "metric_value": disc_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
```

```
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30*40+1, 7))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac6b0fa4.py", line 60, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac6b0fa4.py", line 42, in run_trial
    circuit = generate_ac0_circuit(n, depth)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac6b0fa4.py", line 20, in generate_ac0_
    subcircuits = [generate_ac0_circuit(n // 2, depth - 1) for _ in range(2)]
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac6b0fa4.py", line 20, in generate_ac0_
    subcircuits = [generate_ac0_circuit(n // 2, depth - 1) for _ in range(2)]
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac6b0fa4.py", line 20, in generate_ac0_
    subcircuits = [generate_ac0_circuit(n // 2, depth - 1) for _ in range(2)]
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 1 more time]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac6b0fa4.py", line 21, in generate_ac0_
    return [subcircuits[0][i] ^ subcircuits[1][i] for i in range(n)]
            ~~~~~~~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with IndexError during recursion, preventing data collection. Pre-registered support condition cannot be evaluated. | next: Fix the recursive circuit generation to handle base cases properly and re-run tests with increased seed coverage

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113642
2	propose	ollama_remote	qwen3:8b	0	0	73385
3	preregistration	ollama_remote	qwen3:8b	0	0	24291
4	novelty	ollama_remote	qwen3:8b	0	0	20834
5	novelty	ollama_remote	qwen3:8b	0	0	14864
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15523
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8412
8	judge	ollama_remote	qwen3:8b	0	0	20973

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 291925 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/dd369e80c960.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dd369e80c960.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/dd369e80c960.tar.gz` (if generated)